

FINAL PROJECT DOCUMENTATION

Student Information

- **Name:** S.Surya Gangadhar
 - **Batch:** G-29 FSD
 - **College / Institution:** Gayatri vidya parishad college of engineering
 - **Course / Program:** Full Stack web development using MERN
 - **Trainer / Mentor Name:** Prassana kundu
 - **Submission Date:** 25-04-2026
-

Project Title

RENTAL MANAGEMENT

Live Project Link

<https://rental-management-frontend-kekv.onrender.com>

GitHub Repository

BACKEND - <https://github.com/Surya9908virat/Rental-Management-backend.git>

FROTEXT - <https://github.com/Surya9908virat/Rental-Management-Frontend.git>

Project Overview

The application is a **Rental / Property Management System** that allows users to manage properties, leases, tenants, and related operations through a centralized dashboard.

It provides features such as:

- Viewing and managing property listings
- Tracking leases and tenant details

- Dashboard analytics with charts and statistics
- Role-based navigation (e.g., landlord, tenant, admin)

Additionally, the project focuses on **responsive design**, ensuring the application works smoothly across **mobile, tablet, desktop, and split-screen devices** using Tailwind CSS.

The primary target users are:

- **Landlords / Property Owners** – to manage properties and tenants
 - **Tenants** – to view lease details and related information
-

Idea & Motivation

The idea came from observing that many property management systems are **not mobile-friendly** and are difficult to use on smaller devices.

Since modern users frequently access applications via smartphones, I wanted to create a system that provides a **seamless experience across all screen sizes**.

I chose this project because:

- It solves a **real-world problem** in property and rental management
 - It allowed me to apply **frontend development skills** using React, Tailwind CSS, and responsive design techniques
 - It helped me understand **UI/UX optimization**, especially for mobile devices
 - It demonstrates practical implementation of **modern web development practices**
-

Problem Statement

The project solves the problem of **inefficient and unorganized communication and management between landlords and tenants**.

In traditional systems:

- Rent tracking is often manual or scattered
- Maintenance requests are not properly documented
- Communication happens through calls or messages, leading to confusion
- Important documents like lease agreements are not centrally stored

RentWise provides a **centralized digital platform** where all these activities—rent payments, maintenance requests, messaging, and document management—are handled efficiently in one place.

This problem is faced by:

- **Landlords / Property Owners** – struggle to manage multiple tenants, track rent, and handle maintenance requests
- **Tenants** – face delays in reporting issues, lack transparency in request status, and difficulty accessing lease documents
- **Small building/property managers** – who do not have access to large-scale property management software

This problem is important because:

- Poor management can lead to **financial losses** (missed or delayed rent payments)
- Lack of communication causes **tenant dissatisfaction and disputes**
- Untracked maintenance requests can result in **property damage over time**
- Manual systems are **time-consuming, error-prone, and inefficient**

By solving these issues, RentWise improves:

- **Efficiency** in property management
 - **Transparency** between landlords and tenants
 - **User experience** through real-time updates and communication
 - **Overall reliability** of managing rental properties
-

Solution & Approach

The application solves inefficiencies in landlord-tenant management through the following features:

- **Centralized Dashboard**
Landlords and tenants get role-based dashboards to manage all activities in one place.
 - **Property & Lease Management**
Landlords can add properties, assign tenants, and upload lease agreements (PDFs).
 - **Rent Payment System (Mock)**
Tenants can view due dates and make payments, while landlords can track payment status.
 - **Maintenance Request System**
Tenants can submit issues with images/videos, and landlords can update request status (submitted → in-progress → resolved).
 - **Real-Time Messaging (WebSockets)**
Enables direct communication between landlord and tenant without relying on external apps.
 - **Live Status Updates**
Maintenance request updates are pushed instantly to users using WebSockets.
 - **File Upload System**
Supports uploading lease documents and maintenance proof files (images/videos).
 - **Responsive Design**
Works seamlessly on mobile, tablet, and desktop using Tailwind CSS.
-

• System Workflow

The system follows a structured workflow:

1. **User Authentication**
 - Users log in as either landlord or tenant
 - Role-based access is applied
2. **Landlord Workflow**
 - Adds property details
 - Creates lease and assigns tenant
 - Uploads lease document

- Monitors rent payments and maintenance requests
 - Updates request status
- 3. **Tenant Workflow**
 - Logs in and views lease details
 - Checks rent due date and makes payment
 - Submits maintenance requests with files
 - Tracks request status in real-time
- 4. **Maintenance Flow**
 - Tenant submits request → status = "submitted"
 - Landlord updates → "in-progress"
 - After completion → "resolved"
 - Updates are reflected instantly
- 5. **Messaging Flow**
 - Tenant ↔ Landlord communication via chat
 - Messages are stored and updated in real-time
- **Any Unique or Advanced Logic Used**
 - **Role-Based Access Control (RBAC)**
Ensures users only see features relevant to their role (landlord or tenant).
 - **WebSocket Integration**
Enables:
 - Real-time chat
 - Instant maintenance status updates
 - **Dynamic Navigation System**
Navigation menus change based on user role and device (mobile/desktop responsive design).
 - **File Handling System**
Efficient upload and retrieval of:
 - Lease documents (PDFs)
 - Maintenance images/videos
 - **Responsive UI Architecture**
Uses Tailwind CSS grid and flex utilities to adapt layouts across screen sizes.
 - **Scalable Database Design**
Structured collections (Users, Properties, Leases, Payments, MaintenanceRequests, Messages) ensure efficient data handling.

Modules / Features

Break down your project into modules:

- **Authentication Module**
 - User Registration (Tenant / Landlord)
 - Secure Login & Logout
 - JWT-based Authentication
 - Role-based Access Control
- **User Management**

- Tenant Profile Management
- Landlord Profile Management
- Update Personal Information
- User Dashboard Access
- **Property Management Module**
 - Add New Properties
 - Edit Property Details
 - Delete Property Listings
 - View Available Rental Properties
- **Lease Management Module**
 - Create Lease Agreements
 - Manage Tenant Leases
 - View Active Lease Details
 - Lease Status Tracking
- **Payment Management Module**
 - Rent Payment Tracking
 - Payment History Records
 - Manage Incoming Payments
 - Payment Status Updates
- **Maintenance Management Module**
 - Submit Maintenance Requests
 - Track Request Status
 - Update Maintenance Details
 - Request History Management
- **Messaging / Real-Time Features**
 - Real-Time Chat Between Tenant and Landlord
 - Instant Message Updates using Socket.IO
 - Notification Support
- **API Layer**
 - RESTful API Integration
 - CRUD Operations
 - Secure Backend Endpoints
 - JSON Data Handling
- **Admin / Dashboard Panel**
 - Tenant Dashboard
 - Landlord Dashboard
 - Property Overview
 - Lease & Payment Monitoring



Tech Stack

Frontend

- HTML
- CSS
- JavaScript
- React.js

Backend

- Node.js
- Express.js

Database

- MongoDB

Tools / Services

- GitHub
 - VS Code
 - Postman
 - Socket.IO
 - JWT Authentication
 - MongoDB Atlas
-

API Documentation

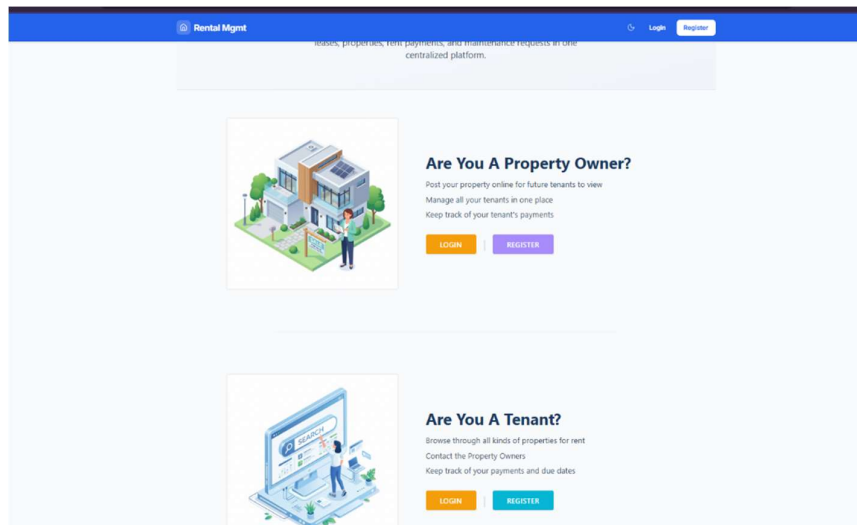
List all APIs used or created in your project:

Method	Endpoint	Description
GET	/api/properties	Fetch all available rental properties
GET	/api/properties/:id	Fetch detailed information about a specific property
POST	/api/auth/register	Register a new tenant or landlord account
POST	/api/auth/login	Authenticate user login credentials
GET	/api/tenant/dashboard	Retrieve tenant dashboard details
GET	/api/landlord/dashboard	Retrieve landlord dashboard details
GET	/api/payments	Fetch payment history records
POST	/api/payments	Process and store a new rent payment
GET	/api/maintenance	Retrieve maintenance requests
POST	/api/maintenance	Submit a new maintenance request
PUT	/api/maintenance/:id	Update maintenance request status
DELETE	/api/maintenance/:id	Delete a maintenance request
GET	/api/leases	Retrieve lease agreement details
POST	/api/leases	Create a new lease agreement
PUT	/api/leases/:id	Update lease information
DELETE	/api/leases/:id	Remove a lease agreement
GET	/api/messages	Fetch messages between tenant and landlord
POST	/api/messages	Send a new message
GET	/api/profile	Retrieve user profile information
PUT	/api/profile	Update user profile details

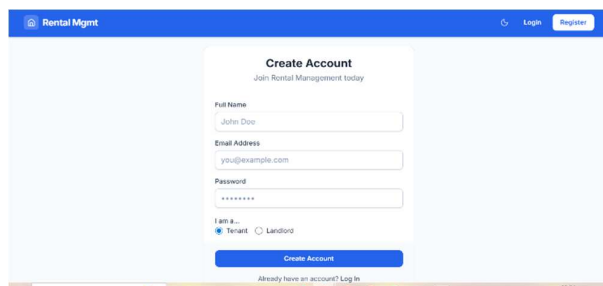
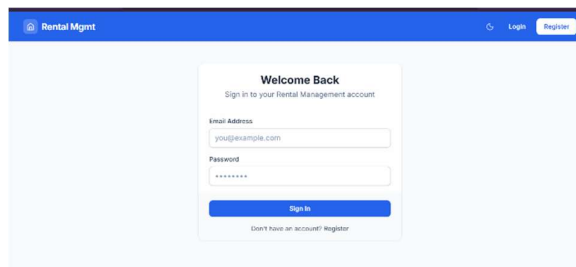
Method	Endpoint	Description
POST	/api/properties	Add a new property listing
PUT	/api/properties/:id	Edit property details
DELETE	/api/properties/:id	Delete a property listing

Project Screenshots

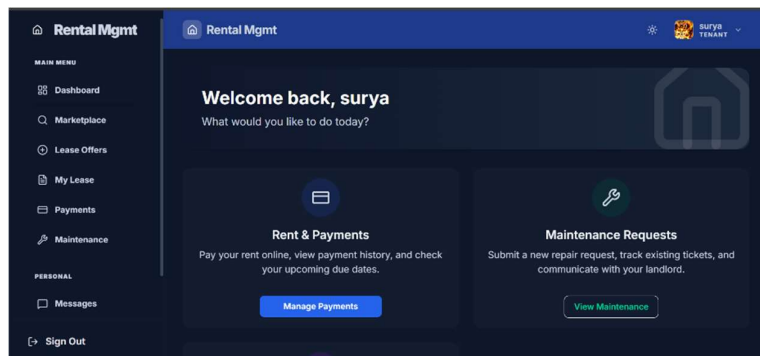
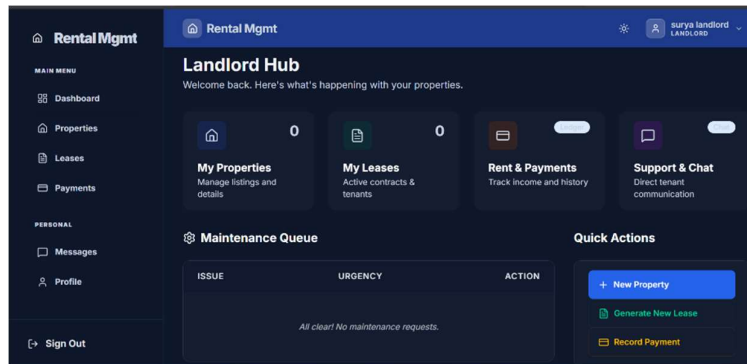
Home Page



Login / Signup



Main Features / Dashboard



Challenges Faced

• Technical Challenges

- Implementing secure authentication and role-based access control for tenants and landlords
- Integrating frontend and backend APIs efficiently
- Managing real-time messaging using Socket.IO
- Handling database relationships between users, properties, leases, and payments
- Ensuring responsive design across different devices

• Logical Problems

- Designing proper property allocation and lease management flow
- Managing payment tracking and status updates accurately
- Handling maintenance request workflows between tenants and landlords
- Preventing unauthorized access to protected routes and data

• Bugs and Fixes

- Fixed login authentication issues caused by incorrect token validation
- Resolved API response delays by optimizing backend routes
- Corrected database schema errors in MongoDB collections
- Fixed UI alignment and responsive layout issues
- Solved real-time messaging synchronization problems in chat functionality
- Debugged routing and navigation errors in React components



Learnings & Outcomes

• Skills Gained

- Full Stack Web Development
- Frontend UI Design and Responsive Development
- Backend API Development using Node.js and Express.js
- Database Management with MongoDB
- Authentication and Authorization Implementation
- Real-Time Communication using Socket.IO
- Debugging and Problem-Solving Skills
- GitHub Version Control and Project Management

• Concepts Learned

- RESTful API Architecture
- CRUD Operations
- Role-Based Access Control
- Client-Server Communication
- Database Schema Design
- JWT Authentication
- State Management and Routing
- Real-Time Data Handling

• Real-World Understanding

- Understanding rental property management workflows
 - Managing tenants, landlords, leases, and payments digitally
 - Importance of secure user authentication and data protection
 - Handling real-time communication in web applications
 - Developing scalable and user-friendly applications
 - Experience working on industry-level project structure and deployment concepts
-



Future Scope / Improvements

• New Features

- Online rent payment gateway integration
- Email and SMS notifications for payments and maintenance updates
- Advanced property search and filtering options
- Document upload for lease agreements and verification
- Mobile application support for tenants and landlords

• Scalability Improvements

- Microservices-based architecture implementation
- Cloud deployment using AWS or Azure
- Load balancing and distributed database support
- Multi-property and multi-location management support

• Performance Optimizations

- Database query optimization for faster response time
- API caching implementation
- Lazy loading and frontend optimization
- Improved real-time communication efficiency using Socket.IO

- **AI Integrations (Future Scope)**

- AI chatbot for tenant support and query handling
 - Smart property recommendation system
 - Predictive maintenance request analysis
 - AI-based rent prediction and analytics system
-



Developer Details

- **Name:** S.Surya Gangadhar
 - **Role:** Full Stack
 - **Email:** swamireddysurya@gmail.com
 - **LinkedIn / Portfolio:** https://www.linkedin.com/in/swamireddy-surya-gangadhar-b70762356?utm_source=share&utm_campaign=share_via&utm_content=profile&utm_medium=android_app
-



Self Introduction

Write a brief professional introduction:

I am Swamireddy Surya Gangadhar, a passionate and motivated Full Stack Development student with a strong interest in web application development and modern technologies. I have worked on projects like the Rental Management System and Restaurant Management System, which helped me gain practical experience in designing and developing real-world applications.

- **Background**

- Pursuing technical education with a focus on software and web development
- Completed training and internship in Full Stack Development
- Hands-on experience in frontend and backend technologies

- **Skills**

- HTML, CSS, JavaScript, React.js
- Node.js, Express.js
- MongoDB Database Management
- REST API Development
- GitHub and Version Control
- Problem Solving and Debugging
- Responsive Web Design

- **Career Goals**

My goal is to become a skilled Full Stack Developer and contribute to innovative software solutions. I aim to continuously improve my technical knowledge, work on real-world projects, and build scalable applications that solve practical problems while growing professionally in the IT industry.

Conclusion

The Rental Management System is a full stack web application developed to simplify and digitalize the process of managing rental properties, tenants, leases, payments, and maintenance requests. The project provides separate functionalities for tenants and landlords through secure authentication and user-friendly dashboards.

This system helps improve efficiency by reducing manual work, organizing rental records digitally, and enabling smooth communication between tenants and landlords. Features such as payment tracking, maintenance management, and real-time messaging make the application practical and useful for real-world rental management scenarios.

Through this project, I gained valuable experience in full stack development, API integration, database management, authentication, and real-time application development. The project strengthened my technical skills and provided practical understanding of building scalable web applications.

Acknowledgment (Optional)

I would like to express my sincere gratitude to my trainers, mentors, and everyone who supported me throughout the development of this project.

• Trainer

- Prasanna Kundu Sir

Their valuable guidance, continuous support, and encouragement helped me improve my technical skills and successfully complete this project.

• Resources / References

- MDN Web Docs
- MongoDB Documentation
- Node.js & Express.js Documentation
- React.js Official Documentation
- GitHub
- Stack Overflow
- YouTube Technical Tutorials

I am thankful to all the learning platforms and resources that contributed to my understanding and project development journey.
